Universidad Nacional de San Agustín de Arequipa

Escuela Profesional de Ciencia de la Computación

Curso: Ingeniería de Software II



LABORATORIO 04

xUnit y Cobertura de Código

Docente: DSc. Edgar Sarmiento Calisaya

Alumno: Cristhian Luis Taipe Huanca

Fecha: 23 / 10 / 2026

1. Introducción

El presente informe documenta el Laboratorio 04 del curso Ingeniería de Software II, cuyo objetivo es aplicar pruebas de caja blanca (whitebox testing) sobre el algoritmo de ordenamiento bubbleSort. A partir del código fuente se construye el grafo de flujo de control, se calcula la complejidad ciclomática y se derivan los caminos de ejecución independientes. Cada camino se traduce en un caso de prueba que luego se automatiza con el framework xUnit correspondiente a Python (módulo unittest / PyUnit), midiendo finalmente la cobertura de código alcanzada con la herramienta Coverage.py.

2. Objetivos

- Aplicar técnicas de prueba de caja blanca: grafo de flujo, complejidad ciclomática y caminos independientes.
- Implementar casos de prueba unitarios con el framework xUnit (unittest/PyUnit).
- Emplear setUp y tearDown, Test Cases y Assertions siguiendo el patrón Arrange–Act–Assert.
- Generar el reporte de ejecución de pruebas y el reporte de cobertura de código.

3. Descripción del problema

Un desarrollador creó un programa que lee una lista de valores enteros y los ordena en forma ascendente mediante el método de la burbuja (bubbleSort). El pseudocódigo original es el siguiente:

Pseudocódigo (enunciado)

```
bubbleSort(int vet[]){
  integer i, j, aux;
  for (i = tamanho(vet); i >= 2; i--){
    for (j = 1; j <= i-1; j++){
      if (vet[j] > vet[j+1]){
        aux = vet[j];
        vet[j] = vet[j+1];
        vet[j+1] = aux;
      }
    }
  }
}
```

Para la implementación se eligió el lenguaje Python (paradigma orientado a objetos en las pruebas) y se separó el código de implementación (bubble_sort.py) del código de pruebas (test_bubble_sort.py).

4. Especificación de casos de prueba (caja blanca)

4.1 Diagrama de flujo de control

El siguiente grafo de flujo de control (CFG) representa las sentencias y decisiones del método bubble_sort. Se identifican tres nodos de decisión: el bucle externo (i ≥ 2), el bucle interno (j ≤ i-1) y la comparación de intercambio (vet[j] > vet[j+1]).

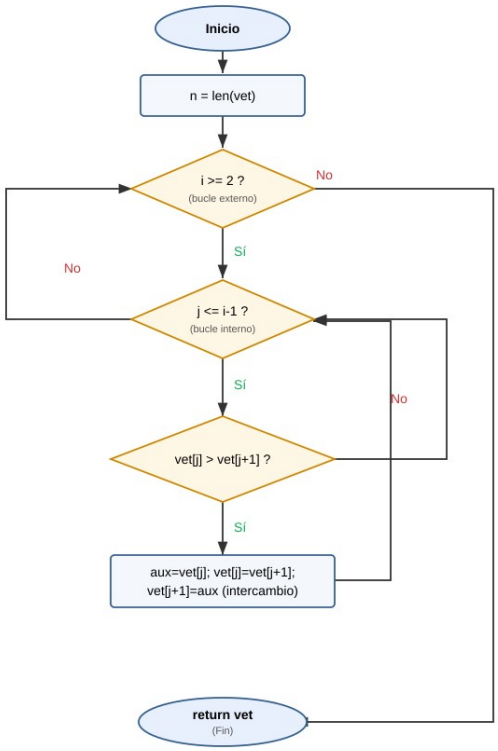


Figura 1. Grafo de flujo de control del método bubble_sort.

4.2 Complejidad ciclomática

A partir del grafo se contabilizan N = 7 nodos, E = 9 aristas y P = 3 nodos predicado (decisiones). La complejidad ciclomática V(G) se calcula mediante las tres fórmulas equivalentes:

Fórmula	Cálculo	V(G)
V(G) = E − N + 2	9 − 7 + 2	4
V(G) = P + 1	3 + 1	4

Fórmula	Cálculo	V(G)
Regiones del grafo	4 regiones	4

El valor obtenido es **V(G) = 4**, por lo que existen 4 caminos de ejecución independientes y se requieren al menos 4 casos de prueba para cubrir la lógica de decisión del algoritmo.

4.3 Caminos de ejecución independientes

Aplicando una estrategia de cobertura de decisiones + condición simple, el conjunto base de caminos independientes es:

- Camino 1: la lista tiene menos de 2 elementos → el bucle externo no se ejecuta (i ≥ 2 falso).
- Camino 2: se entra a ambos bucles pero la condición de intercambio es siempre falsa (lista ya ordenada).
- Camino 3: la condición de intercambio es verdadera al menos una vez (se ejecuta el swap).
- Camino 4: múltiples pasadas con varios intercambios (caso general con lista desordenada).

4.4 Casos de prueba identificados

ID CP	Camino en el Grafo	Valores de Prueba	Resultado Esperado
CP1	Inicio → (i≥2: No) → Fin	[42]	[42]
CP2	Inicio → i≥2:Sí → j≤i-1:Sí → if:No → ... → Fin	[10, 20, 30]	[10, 20, 30]
CP3	Inicio → i≥2:Sí → j≤i-1:Sí → if:Sí → swap → ... → Fin	[20, 10]	[10, 20]
CP4	Recorrido completo con varias pasadas e intercambios	[5, 1, 4, 2, 8]	[1, 2, 4, 5, 8]

5. Implementación con xUnit

Se utilizó el framework **unittest (PyUnit)**, la implementación del estándar xUnit para Python, y la herramienta **Coverage.py** para el reporte de cobertura. La dependencia se gestiona vía requirements.txt (coverage) en un entorno virtual.

5.1 Código bajo prueba (bubble_sort.py)

bubble_sort.py

```
def bubble_sort(vet: list[int]) -> list[int]:
    n = len(vet)

    for i in range(n, 1, -1):
        for j in range(0, i - 1):
            if vet[j] > vet[j + 1]:
                aux = vet[j]
                vet[j] = vet[j + 1]
                vet[j + 1] = aux

    return vet
```

5.2 Suite de pruebas (test_bubble_sort.py)

La suite emplea setUp y tearDown, cuatro Test Cases (uno por camino independiente) y Assertions siguiendo el patrón Arrange–Act–Assert:

test_bubble_sort.py

```
import unittest
from bubble_sort import bubble_sort

class TestBubbleSort(unittest.TestCase):

    def setUp(self):          # Arrange global (antes de cada prueba)
        self.resultado = None

    def tearDown(self):       # Limpieza (despues de cada prueba)
        self.resultado = None

    def test_cp1_lista_un_elemento(self):
        lista = [42]; esperado = [42]              # Arrange
        self.resultado = bubble_sort(lista)        # Act
        self.assertEqual(self.resultado, esperado) # Assert

    def test_cp2_lista_ya_ordenada(self):
        lista = [10, 20, 30]; esperado = [10, 20, 30]
        self.resultado = bubble_sort(lista)
        self.assertEqual(self.resultado, esperado)

    def test_cp3_un_solo_intercambio(self):
        lista = [20, 10]; esperado = [10, 20]
        self.resultado = bubble_sort(lista)
        self.assertEqual(self.resultado, esperado)

    def test_cp4_lista_desordenada(self):
        lista = [5, 1, 4, 2, 8]; esperado = [1, 2, 4, 5, 8]
```

```
        self.resultado = bubble_sort(lista)
        self.assertEqual(self.resultado, esperado)

if __name__ == "__main__":
    unittest.main()
```

6. Resultados

6.1 Reporte de ejecución de pruebas

Las cuatro pruebas se ejecutaron con unittest en modo verboso. El resultado fue PASÓ (OK) en los cuatro casos:

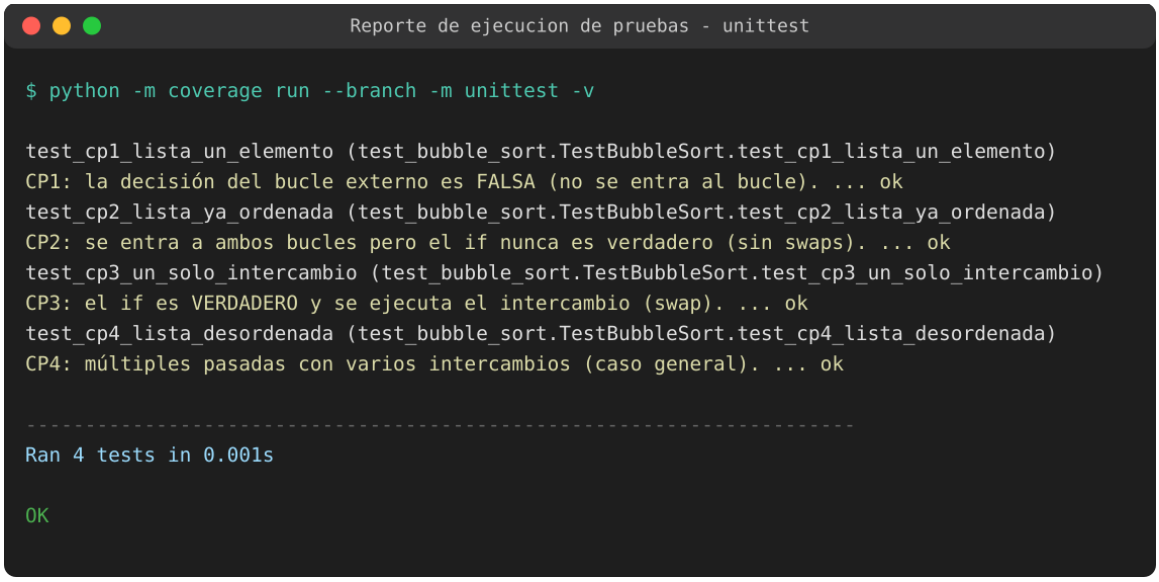


Figura 2. Reporte de ejecución de pruebas (4/4 OK).

ID CP	Caso de prueba	Resultado obtenido	Estado
CP1	Lista de un elemento [42]	[42]	PASÓ
CP2	Lista ya ordenada [10, 20, 30]	[10, 20, 30]	PASÓ
CP3	Un solo intercambio [20, 10]	[10, 20]	PASÓ
CP4	Lista desordenada [5, 1, 4, 2, 8]	[1, 2, 4, 5, 8]	PASÓ

6.2 Reporte de cobertura de código

Con Coverage.py (en modo *--branch*) se obtuvo **100% de cobertura** de sentencias y de ramas sobre bubble_sort.py: las 9 sentencias y las 6 ramas fueron ejecutadas por los casos de prueba.

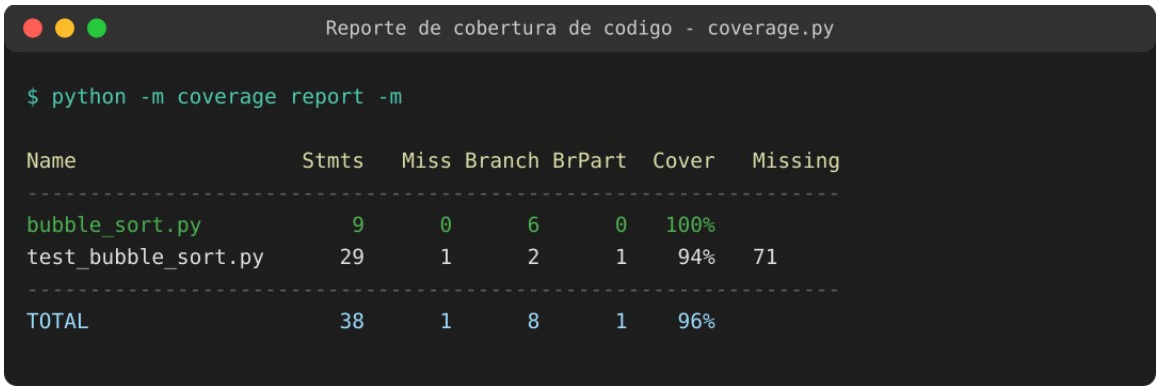


Figura 3. Resumen de cobertura por terminal (coverage report -m).

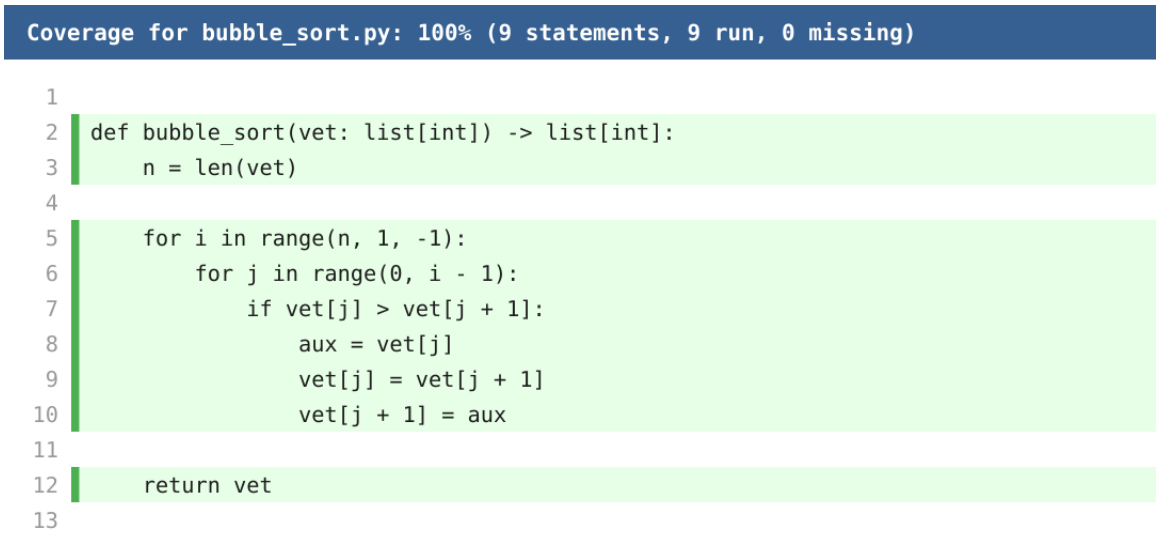


Figura 4. Vista de cobertura línea por línea (reporte HTML de Coverage.py): todas las líneas ejecutadas.

Archivo	Sentencias	No cubiertas	Ramas	Cobertura
bubble_sort.py	9	0	6	100%

Nota: el archivo de pruebas test_bubble_sort.py reporta 94% únicamente por la guarda "if __name__ == '__main__'", línea no ejecutada al correr bajo el runner; el código bajo prueba alcanza 100%.

7. Conclusiones

- La complejidad ciclomática V(G) = 4 determinó exactamente el número de casos de prueba necesarios para cubrir la lógica de decisión.
- Los cuatro casos derivados de los caminos independientes se automatizaron con xUnit (unittest) y todos pasaron (4/4 OK).

• La cobertura de código alcanzó el 100% de sentencias y ramas en bubble_sort.py, confirmando que los caminos independientes ejercitan toda la lógica.

• El flujo caja blanca (grafo → complejidad → caminos → casos) resultó efectivo y reproducible mediante herramientas abiertas (unittest y Coverage.py).

8. Entregables y referencias

• Especificación de casos de prueba: grafo de flujo, complejidad ciclomática y tabla CP1–CP4 (secciones 4.1–4.4).

• Implementación: bubble_sort.py y test_bubble_sort.py (proyecto a publicar en GitHub).

• Reporte de ejecución de pruebas: Figura 2 (unittest -v).

• Reporte de cobertura de código: Figuras 3 y 4 y reporte HTML en la carpeta htmlcov/ (Coverage.py).

• Documentación: unittest (docs.python.org/3/library/unittest.html), Coverage.py (coverage.readthedocs.io).